🏫

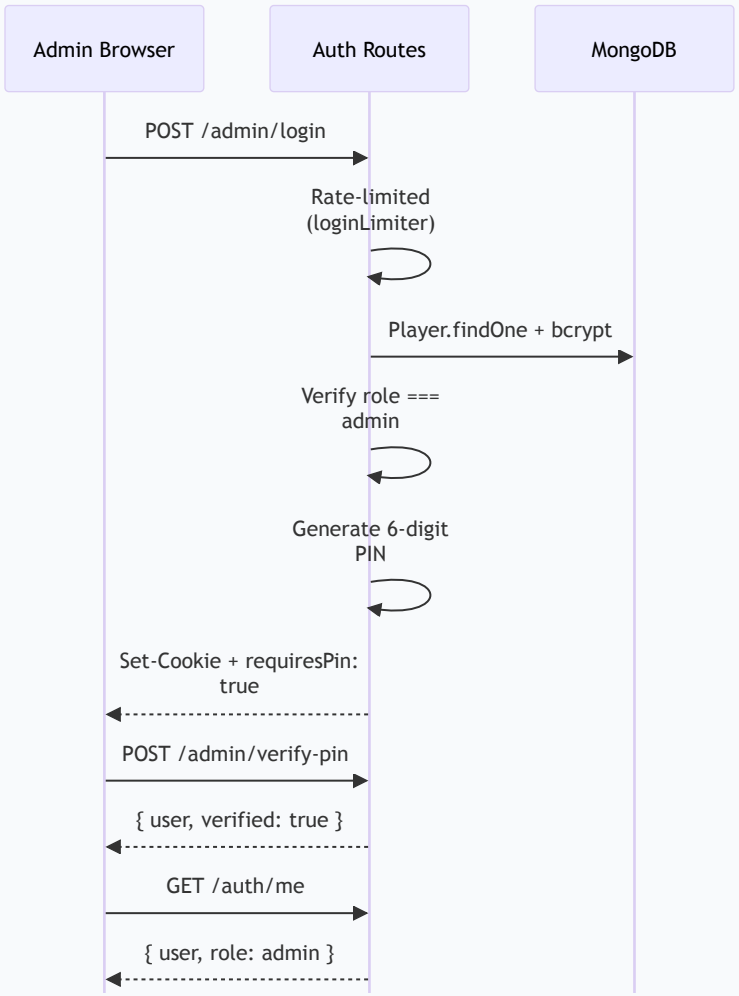
Academy (Admin) Architecture

Complete admin control plane — tournaments, staff, diagnostics, and operations

AceTrack Architecture Documentation • Generated 12 June 2026

1. Admin Authentication Flow

Admin login is web-only with a two-factor PIN verification step for enhanced security.



Detailed Flow Breakdown

1 Web-Only Login Gate

Admin login is strictly web-only (platform: "web" required). This prevents regular mobile users from encountering the admin login flow. Rate-limited by loginLimiter middleware.

```
POST /api/v1/admin/login { username, password, platform: "web" }
```

2 Credential Verification

Server queries Player collection by username, verifies the password hash with bcrypt, and confirms the user has role: "admin". Non-admin accounts are rejected with 403.

3 **2FA PIN Generation**

A 6-digit PIN is generated and stored in the session. The initial JWT cookie is issued with requiresPin: true, meaning the client must complete PIN verification before accessing admin features.

4 **PIN Verification**

Admin enters the PIN (delivered via a trusted channel). Server validates the PIN matches the session, then upgrades the JWT to a fully-verified state.

```
POST /api/v1/admin/verify-pin { pin: "123456" }
```

5 **Session Management**

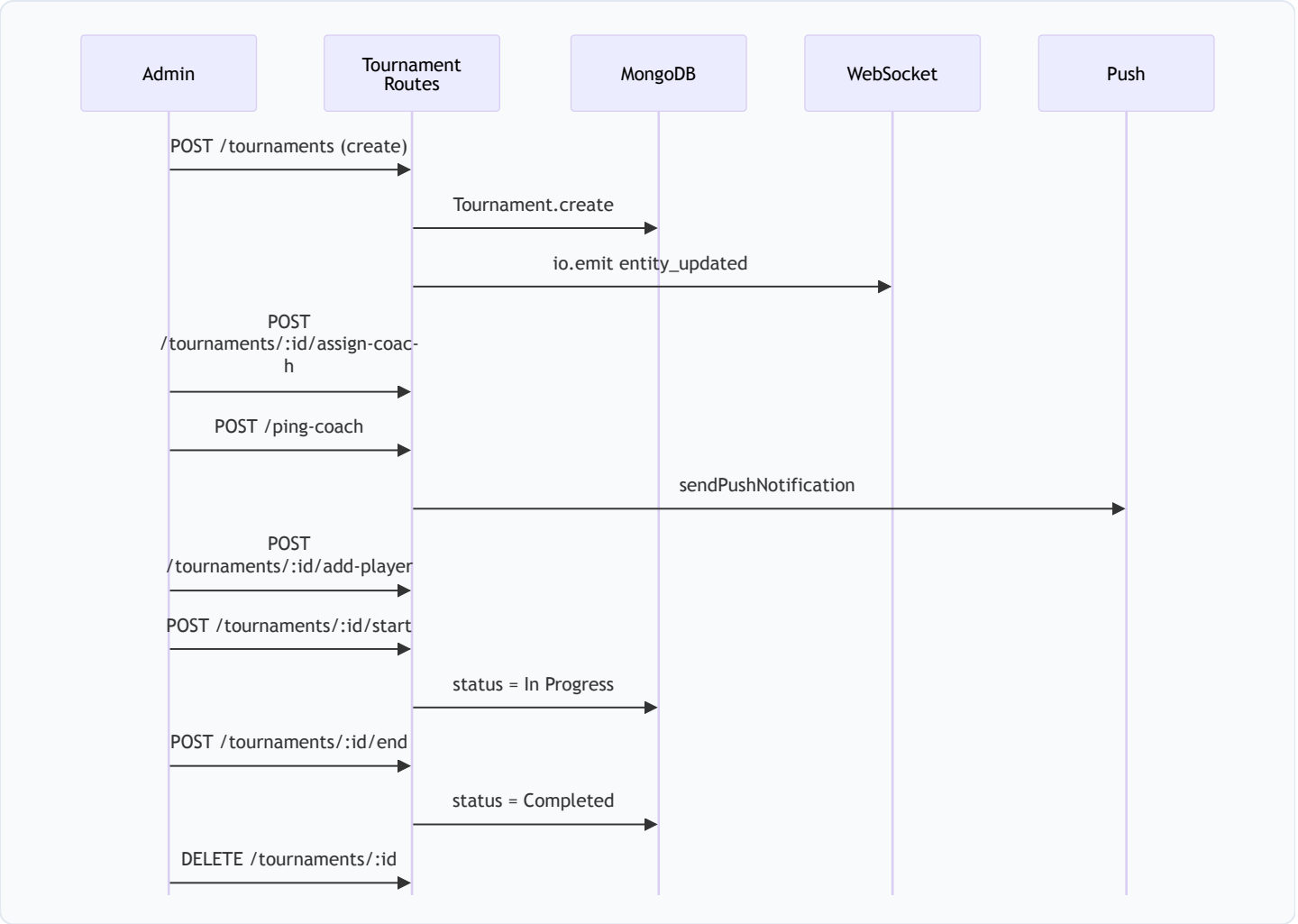
Subsequent requests use the verified JWT cookie. The /auth/me endpoint returns user data with role for client-side UI rendering. Logout clears the HttpOnly cookie.

API Endpoints

ENDPOINT	METHOD	PURPOSE
`/admin/login`	POST	Admin login (web-only)
`/admin/verify-pin`	POST	2FA PIN verification
`/auth/me`	GET	Session check (JWT cookie)
`/logout`	POST	Clear auth cookie
`/admin/restore-last-state`	POST	Restore previous AppState snapshot

2. Tournament Full Lifecycle

The admin controls the complete tournament lifecycle: creation → coach assignment → player management → start → end → deletion.



Detailed Flow Breakdown

1 Create Tournament

Admin provides title, sport, date/time, location, max players, cost, sponsor, type (singles/doubles), and category. A unique ID is generated, document created in Tournament collection, and synced to AppState.

```
POST /api/v1/tournaments { title, sport, date, ... }
```

2 Update Tournament

Admin can modify any tournament field before it starts. Changes are broadcast via WebSocket for real-time client updates.

```
PUT /api/v1/tournaments/:id { ...fields }
```

3 Assign Coach

Admin assigns a coach by ID. The system updates the tournament document and may trigger notifications to the coach based on their availability.

```
POST /api/v1/tournaments/:id/assign-coach { coachId }
```

4 Ping Coach

Sends a push notification invitation to the coach. Validates availability first, increments ping counter, and tracks delivery status.

```
POST /api/v1/ping-coach { tournamentId, coachId }
```

5 Manual Player Add

Admin manually adds a player to a tournament using $addToSet to prevent duplicates.

```
POST /api/v1/tournaments/:id/add-player { playerId }
```

6 **Manage Interested Players**

Admin approves or rejects players from the interested list, moving approved players to registeredPlayerIds.

```
POST /tournaments/:id/manage-interested { playerId, action }
```

7 **Start Tournament**

Sets status to "In Progress". Broadcasts update to all connected clients.

```
POST /api/v1/tournaments/:id/start
```

8 **End Tournament**

Sets status to "Completed". Finalizes match results and enables public results page.

```
POST /api/v1/tournaments/:id/end
```

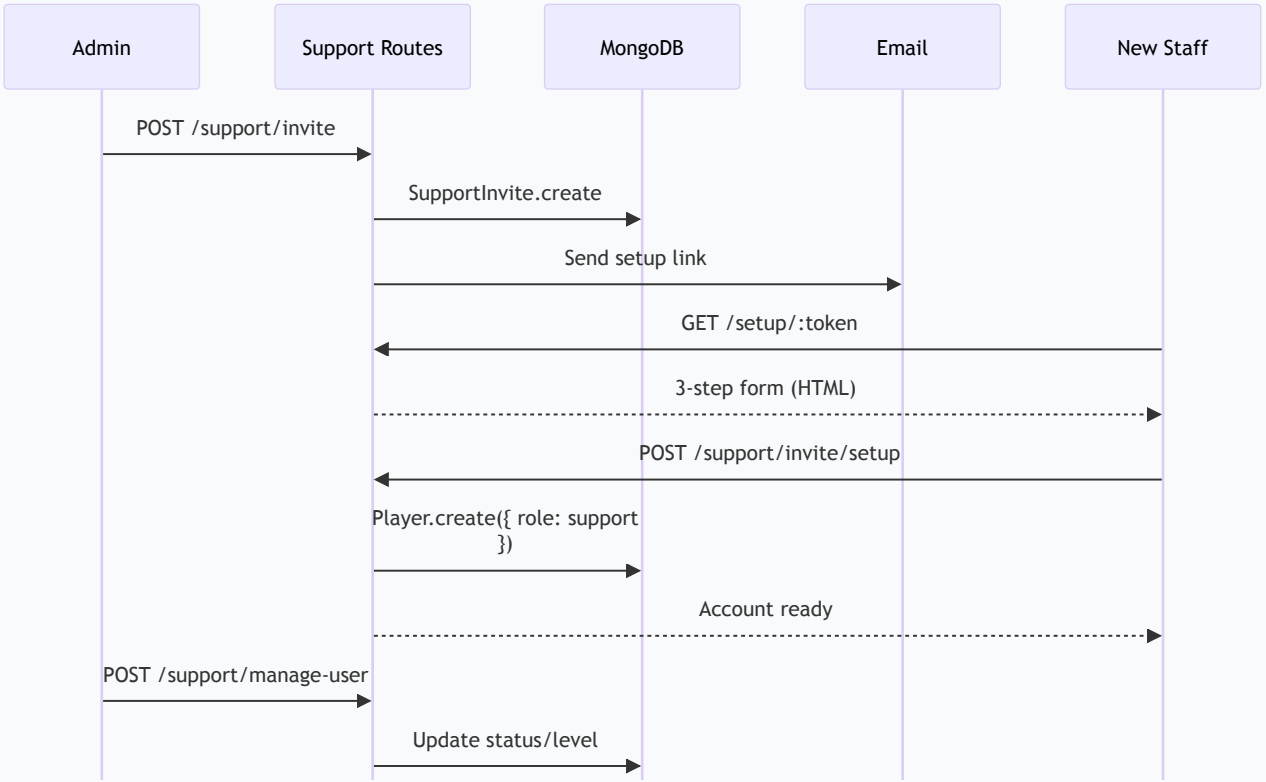
9 **Delete Tournament**

Permanently removes the tournament from both the distinct collection and AppState. Broadcasts deletion via WebSocket.

```
DELETE /api/v1/tournaments/:id
```

3. Support Staff Management

Admin onboards, manages, and monitors support staff through a dedicated invite → setup → management workflow.



Detailed Flow Breakdown

1 **Send Staff Invite**

Admin specifies email, designation, and support level (L1/L2/Manager). A SupportInvite document is created with a unique token and 7-day expiry. An onboarding email with the setup link is dispatched.

```
POST /api/v1/support/invite { email, designation, supportLevel }
```

2 **Staff Onboarding Form**

Staff clicks the link → server-rendered 3-step HTML form: Step 1 (Personal Info: name, phone, address), Step 2 (Government ID upload), Step 3 (Password creation). CSP nonces ensure XSS safety.

3 **Account Finalization**

Staff submits the multipart form. Server creates a Player document with role "support", hashes the password, uploads gov ID to Cloudinary, and marks the invite as "Used".

```
POST /api/v1/support/invite/setup (FormData)
```

4 **Manage Existing Staff**

Admin can activate/deactivate/promote staff members, change support levels, or force password resets.

```
POST /api/v1/support/manage-user { userId, action }
```

5 **Invite Management**

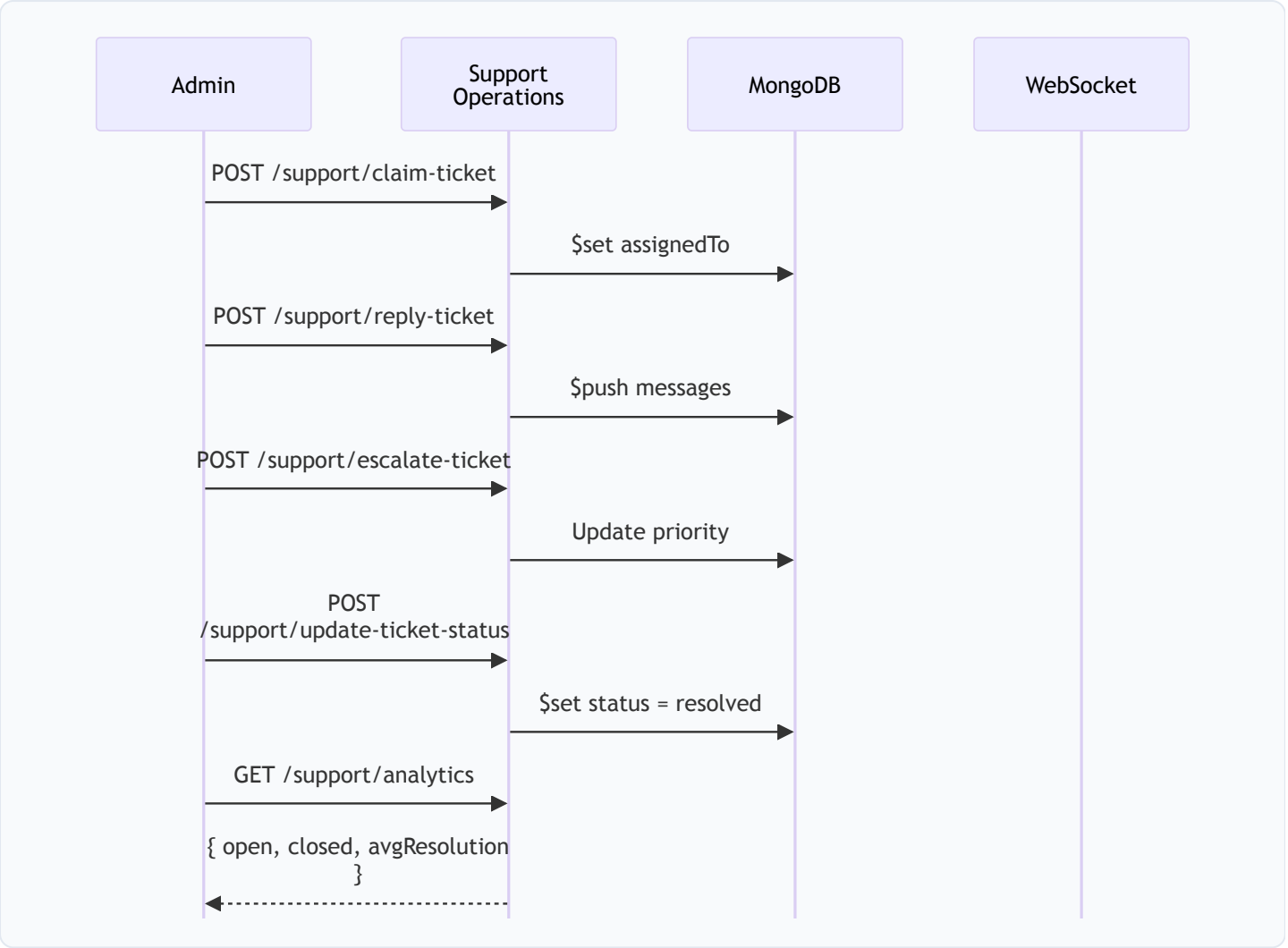
Admin can list all invites, retire unused invites, or resend expired ones with new tokens.

API Endpoints

ENDPOINT	METHOD	PURPOSE
`/support/invite`	POST	Send staff invite
`/support/invites`	GET	List all invites
`/support/invite/expire`	POST	Retire an invite
`/support/invite/resend`	POST	Resend invite email
`/support/invite/setup`	POST	Complete onboarding
`/support/manage-user`	POST	Manage staff status
`/support/force-reset`	POST	Force password reset

4. Support Ticket Operations

Admin has full visibility and control over the support ticket lifecycle — from triage to resolution analytics.



Detailed Flow Breakdown

1 Claim Ticket

Admin or support agent claims an unassigned ticket. Sets assignedTo field and broadcasts update via WebSocket.

```
POST /api/v1/support/claim-ticket { ticketId }
```

2 Reassign Ticket

Admin transfers a ticket to a different agent. Updates assignedTo and notifies the new assignee.

```
POST /api/v1/support/reassign-ticket { ticketId, newAssigneeId }
```

3 Escalate Ticket

Raises the ticket's priority level. Can escalate from L1 → L2 → Manager for complex issues.

```
POST /api/v1/support/escalate-ticket { ticketId, escalateToLevel }
```

4 Reply to Ticket

Send a response to the player (public) or add internal notes (isInternal: true). Messages are pushed to the ticket's thread.

```
POST /api/v1/support/reply-ticket { ticketId, message, isInternal }
```

5 AI Summary

Generate an AI-powered summary of the entire ticket thread for quick context understanding.

```
POST /api/v1/support/ai-summary { ticketId }
```

6 Resolve & Close

Update ticket status through the lifecycle: open → in_progress → resolved → closed. Players can rate the resolution quality.

```
POST /api/v1/support/update-ticket-status { ticketId, status }
```

7 **Analytics Dashboard**

View aggregate support metrics: open/closed ticket counts, average resolution time, tickets per agent, SLA compliance, and trend data.

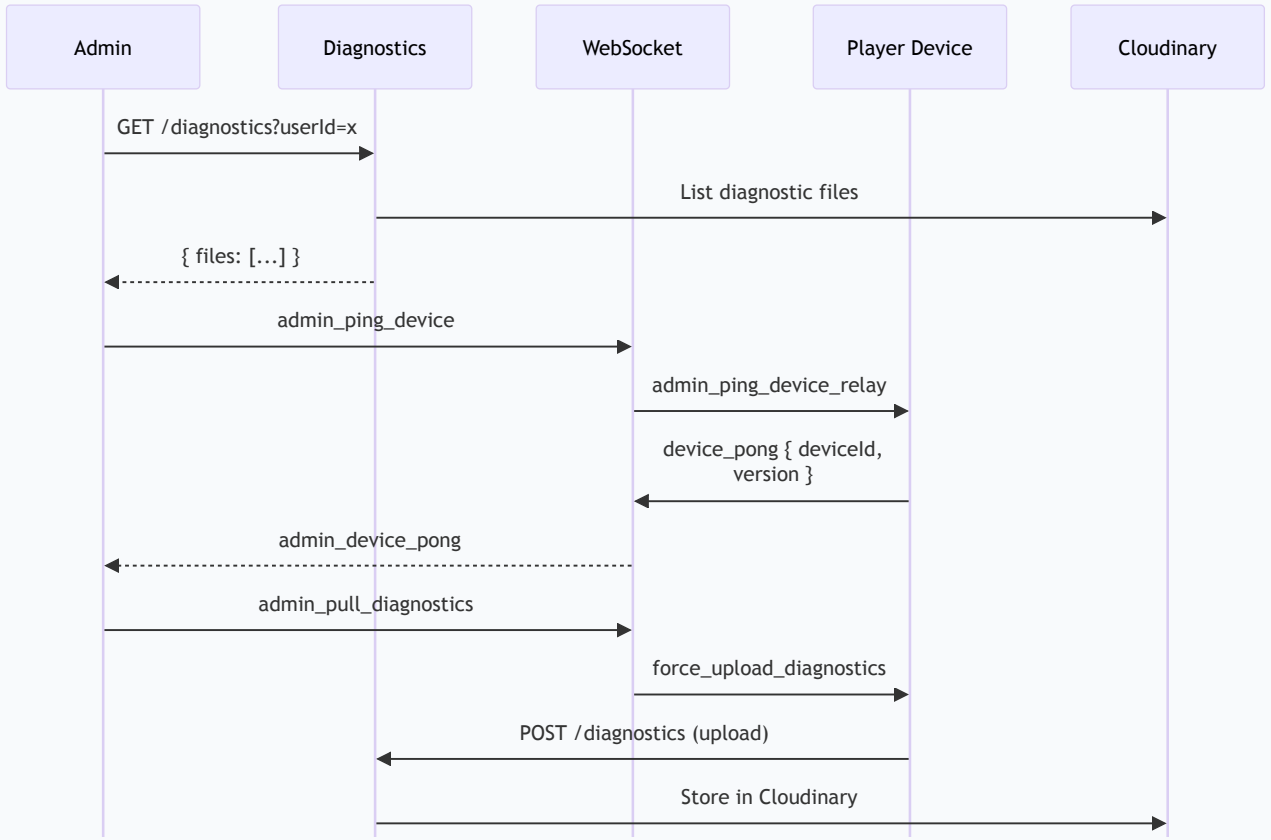
8 **Bulk Transfer**

Transfer all tickets from one agent to another (e.g., during employee offboarding or leave).

```
POST /api/v1/support/transfer-tickets { fromAgentId, toAgentId }
```

5. Diagnostics & Device Management

Admin can remotely query diagnostics, ping devices, and pull logs from player devices via the WebSocket relay.



Detailed Flow Breakdown

1 **List Diagnostic Files**

Admin queries diagnostics for a specific user. Server fetches from both Cloudinary CDN and local filesystem, deduplicates, and sorts by timestamp.

```
GET /api/v1/diagnostics?userId=player123
```

2 **Read Diagnostic File**

Admin opens a specific diagnostic file. Server tries Cloudinary first, falls back to local filesystem if cloud copy doesn't exist.

```
GET /api/v1/diagnostics/:filename
```

3 **Ping Device**

Admin sends a WebSocket ping to check if a specific device is online. The target device responds with its deviceId, name, and app version.

4 **Pull Diagnostics**

Admin remotely triggers a diagnostic upload from a specific device. The device gathers its current DeviceLogger logs and uploads them with a "admin_requested" prefix.

```
WebSocket: admin_pull_diagnostics → force_upload_diagnostics
```

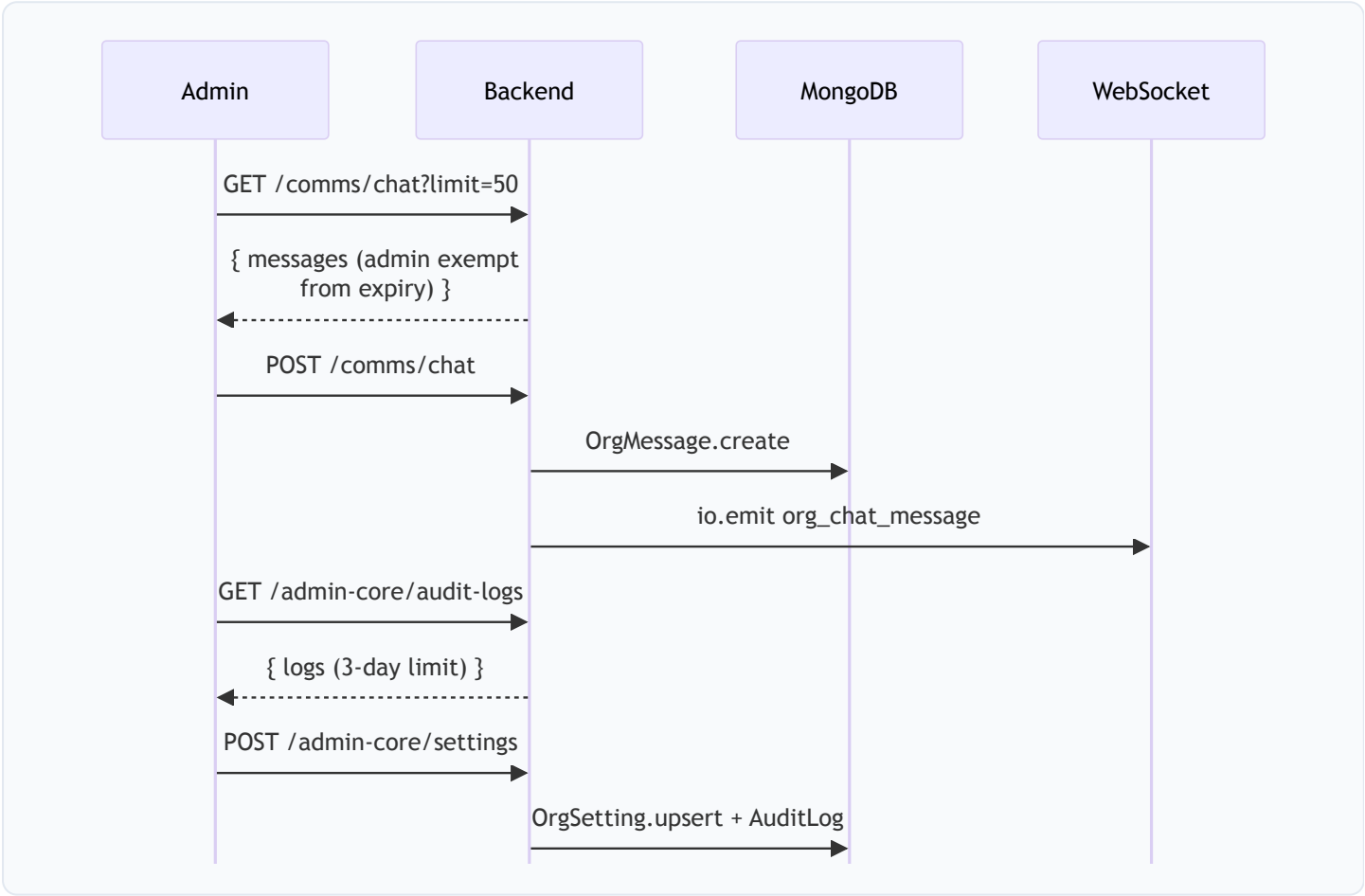
5 **Raw Server Events**

Admin can access raw server-side events from the server_events.jsonl file for deep troubleshooting.

```
GET /api/v1/diagnostics/raw_events
```

6. Org Chat, Audit Logs & Settings

Admin manages internal communications, reviews audit trails, and configures academy-wide settings.



Detailed Flow Breakdown

1 **Org Chat (Admin-Only Features)**

Admin can send messages, upload attachments (via Cloudinary with 7-day expiry), react with emojis, and delete any message. Admin is exempt from the 7-day attachment expiry — can access all files permanently.

2 **Chat Features**

The chat system supports threaded replies (replyTo field), emoji reactions (Map toggle), file attachments, read receipts, and real-time delivery via WebSocket.

3 **Announcements**

Admin can view broadcast announcements that are sent to all staff members.

```
GET /api/v1/comms/announcements
```

4 **Audit Logs**

Admin queries audit logs with date range and search filters. A 3-day limit is enforced without specific search criteria to prevent excessive database load. Results are capped at 200 records.

```
GET /api/v1/admin-core/audit-logs?startDate=...&search=LOGIN
```

5 **Academy Settings**

Admin creates or updates organizational settings (key-value pairs). Every change is logged in the AuditLog with the admin's identity and IP address.

```
POST /api/v1/admin-core/settings { key, value }
```

6 **Team Directory**

Admin views all staff with enriched presence data (device activity, online/offline status, ex-employee detection) and manages reporting hierarchy.

```
GET /api/v1/admin-core/team-directory
```

7 **Security Export**

Admin can export compiled security audit data for compliance review.

```
GET /api/v1/infrastructure/security/export
```